Australian National University

Dr Alex Potanin, Associate Professor (https://potanin.github.io/)
School of Computing, College of Systems and Society

Education: BSc (Mathematics and Computer Science), BSc Hons, PhD, *Victoria University of Wellington, NZ*

Memberships: Member of Engineers Australia, Senior Member of the Association for Computing Machinery, Permanent Member of the IFIP 2.4 Working Group on Software Implementation Technology

Expertise: My research area is *programming languages* and *software verification*. I work on the mathematical principles that define the behaviour of the essential tools that engineers use to craft all the fantastic technology surrounding us today. My contributions to *type systems* around *ownership*, *immutability*, and *capabilities* redefined the *security guarantees* that modern *software engineering* can provide.

Impact: My work over the first decade of my career involved the concepts of ownership and immutability, and how to provide usable language support for both with the help of type parameters. My approach has now been widely adopted by the Rust programming language as "lifetime parameters". In the second decade, I worked on the design and production of a usable and secure programming language called Wyvern that utilises object capabilities and effects. A popular configuration language called CUE is used widely within Alibaba's cloud and service configuration. CUE based its module system design on the Wyvern modules.

After a full-year sabbatical at what was then the Institute for Software Research at Carnegie Mellon University, working with Professor Jonathan Aldrich, I have created a novel general-purpose Wyvern Programming Language designed from the ground up with security and usability as its primary goals. Many students and publications came out of that project over the following decade, including novel ideas for type-specific languages and decidable typing for type members - some of which are reflected in the modern generation of the industrial Scala Programming Language. I am currently working on some ideas for the modern module systems designs based on capabilities, combinations of abstract and algebraic effects, and other programming language design ideas, including for the world of fully verified and secure software targeting embedded and quantum computing systems.

Beyond academia, I have experience in the software industry, including early-stage involvement with startups. I am committed to mentoring the next generation of computer scientists and welcome inquiries from prospective graduate students interested in programming languages, software security, and formal verification.

Significant Contributions to the Field

My research career started as early as my Honours year, when I studied memory relationships between objects in heaps – leading to an eventual publication of our discovery of the scale-free nature of object graphs that has over 300 citations in Google Scholar. My PhD demonstrated that type polymorphism has a deep connection to ownership types, resulting in my two most important papers (each with over 100 citations on Google Scholar) showing how "generic ownership" and "generic immutability" can allow ownership and immutability guarantees to be incorporated "for free" in the existing programming language designs. I was proud to see my research work directly integrated into the design of the Rust Programming Language, only a few years after my publications in OOPSLA 2006 and FSE 2007 (which won the distinguished paper award at ESEC/FSE 2007), that is now taking the world by storm and replacing C as the language of choice for systems programming.

In 2008 – 2012, I obtained a $300,000 Marsden Grant funding on demonstrating how the concepts of ownership and immutability can be unified with an underlying mathematical framework with several additional publications exploring the ownership and immutability implications for secure software development. My ICSE 2013 empirical study and OOPSLA 2010 paper show how essential collections libraries' designs and language designs can be easily tweaked to provide long-lasting security guarantees. I continued to produce other research publications on a variety of topics at the same time, ranging from studying multiple dispatch mechanisms to the usability of GIS and mapping software.

My research career pivoted again in 2013 during my sabbatical where I switched to the fundamentals of general programming language design and security. Our breakthroughs included a novel way of defining "type-specific languages" where any type will have a parser associated with it that allows a mixed language code to be safely type checked and parsed at the same time preventing "SQL injection" and similar attacks. Our paper was the distinguished paper award at ECOOP 2014. We also demonstrated the decidability of type members – a problem that alluded programming language researchers for at least 2 or 3 decades and showed how capabilities can benefit the design of module systems (that this grant builds upon) as well as being the first ones to demonstrate in 2018 how effect systems can be simplified by using capability-like mechanisms.

Meanwhile, my security-related work at VUW resulted in an ECOOP 2017 paper on the "Evil Pickles" showing how large systems can be brought down with the right crafting of user input due to the inherent algorithmically costly processing involved. We won the Distinguished Artifact Award at ECOOP 2017. Furthermore, we were able to secure $50,000 USD Oracle Labs funding to support our secure language research.

More recently, I moved into the area of software verification – spending my 2019/2020 sabbatical that had to be aborted after just 4 months due to COVID-19 in Japan, working at the leading Kyoto University where I taught a graduate course purely around my research outputs so far (https://potanin.github.io/kyoto2020/) and established a new relationship including remote supervision with Atsushi Igarashi – who, together with Naoki Kobayashi, has been developing a verifier that utilises ownership to automatically infer correctness guarantees.

Finally, the breadth of my research goes beyond programming languages and security where we are working with local farmers on a token trading platform to influence pollution mitigation behaviours and balance the economic costs involved. Most recently, we obtained $72,000 NZD funding from Robonomics Network to establish a small network of Nitrate Sensors to support the proposed marketplace platform and we have additional PhD student support funding to continue this research well into the future.

Collaborations

Assistant Professor Liyi Li, Iowa State University, USA
Professor Ina Schaefer, Karlsruhe Institute of Technology, Germany
Professor Jonathan Aldrich, CMU, USA
Professor Atsushi Igarashi, Kyoto University, Japan

Referees

Professor Peter Müller, *ACM Fellow*, ETH Zurich, Switzerland
Professor Steve Blackburn, *ACM Fellow*, ANU/Google DeepMind
Professor Jeff Foster, *ACM Fellow*, Tufts University, USA
Professor David Lo, *ACM Fellow*, Singapore Management University
Professor Anders Møller, Aarhus University, Denmark
Professor Sophia Drossopoulou, Imperial College London, UK
Professor Jan Vitek, Northeastern University, USA

Previous Managers

Professor Tony Hosking, Director of the School of Computing, ANU
Dr Stuart Marshall, former Head of School of Engineering and Computer Science, VUW

Selected Prizes and Awards: Distinguished Paper at the FSE 2007, Distinguished Paper at the ECOOP 2014, Best Paper at FORTE2022, Distinguished Artifact at the ECOOP 2017 and ECOOP 2024, Distinguished Paper at ETAPS/ESOP 2026, and grants as above.

Key Publications and Outputs:

- **Alex Potanin**, Johan Ostlund, Yoav Zibin, Michael D. Ernst. *Immutability*. In D. Clarke et al. (Eds.): Aliasing in Object-Oriented Programming, LNCS 7850, pp. 233-269. Springer, Heidelberg. 2013.
    - This is a particularly special piece of work in the prestigious Lecture Notes on Computer Science (LNCS) series by Springer. Both in terms of the review process and the work involved, this book chapter was essentially a typical top journal article. This work surveys the field as well as consolidates four major papers by the authors, which makes it of significant value. Originally, I was invited to write this chapter because of my research work on immutability and ownership. I took the lead author role, writing almost all the text. I built upon 2 of my own papers and 2 further papers by my co-authors to develop a self-contained survey of immutability and its impact on object-oriented languages in use today. All the chapters in the book underwent a thorough review process by the experts in our field so the quality assurance was both in terms of invitation to write and in terms of the review process. I am very proud of this piece of work, as it is the pinnacle publication summarising my findings because of the now completed Marsden Grant.
- Tobias Runge, Marco Servetto, **Alex Potanin**, Ina Schaefer. *Immutability and Encapsulation for Sound OO Information Flow Control*. ACM Transactions on Programming Languages and Systems. Volume 45, Issue 1, Article No.: 3 pp 1–35.
    - This work demonstrated an insightful application of software verification technique of "correctness by construction" to the widely regarded problem of information flow in software security.
- Isaac Oscar Gariano, Marco Servetto, **Alex Potanin**. *Using Capabilities for Strict Runtime Invariant Checking.* Science of Computer Programming, Volume 224, 2022.
    - This work applied my object capability expertise to show a way to simplify the software verification approach to runtime verification.
- Jens Dietrich, Kamil Jezek, Shawn Rasheed, Amjed Tahir, **Alex Potanin**. *EvilPickles: DoS attacks based on Object-Graph Engineering*. Proceedings of ECOOP2017.
    - This paper discovered a major security flaw in the web servers used by major companies based on Java or .NET that built upon exploiting the data structures (collections) design in most major programming languages. Oracle and Microsoft both accepted our security reports 6 months prior to making this paper available to general public. Other languages, such as Mozilla's Rust, are currently fixing the vulnerability. Together with the next paper this shows my heavy recent involvement in security related research that has an ECOOP 2017 paper session dedicated to Security with these two presentations. This work was further presented at commercial security conferences, such as OWASP NZ.
- Darya Melicher, Yangqingwei Shi, **Alex Potanin**, Jonathan Aldrich. *A Capability-Based Module System for Authority Control*. Proceedings of ECOOP2017.
    - This work outlined a novel way to prevent unauthorised information flow between different parts of a software system. Our approach allowed for the enforcement at a programming language level before the actual execution takes place. This work is the main reason that

Oracle Labs is extremely interested in our approach to the problem and decided to fund our research further.

- Cyrus Omar, Darya Melicher, Ligia Nistor, Benjamin Chung, **Alex Potanin**, and Jonathan Aldrich. *Safely Composable Type-Specific Languages*. Proceedings of ECOOP2014. Pp 105-130. Springer-Verlag.
    - This paper won best paper award as it described a way to safely combine multiple languages together (DSL's) in a type checked, safe manner. This was a ground-breaking approach that received good news coverage:
        - http://sdtimes.com/wyvern-language-works-platforms-interchangeably/
        - http://www.itbusinessedge.com/blogs/integration/wyvern-programming-language-leverages-data-types-to-specify-sublanguages.html
        - https://news.ycombinator.com/item?id=8151432
        - http://classic.slashdot.org/story/14/08/10/2250205
        - http://lambda-the-ultimate.org/node/5018
        - http://www.reddit.com/r/programming/comments/2cx2g7/the_wyvern_programming_language/
        - http://m.habrahabr.ru/post/232975/
        - http://technews.acm.org/archives.cfm?fo=2014-08-aug/aug-11-2014.html#740557
        - http://www.cs.cmu.edu/news/carnegie-mellon-developing-programming-language-accommodates-multiple-languages-same-program
- Marco Servetto, Julian Mackay, **Alex Potanin**, and James Noble. The Billion-Dollar Fix: Safe Modular Circular Initialisation with Placeholders and Placeholder Types. In proceedings of ECOOP2013. Pages 205-229. Springer-Verlag.
    - This work provided a solution to a long-standing circular initialisation problem in programming language design: the ability to initialise mutually dependent objects while guaranteeing immutability after construction.
- **Alex Potanin,** Monique Damitio and James Noble. Are Your Incoming Aliases Really Necessary? Counting the Cost of Object Ownership. In proceedings of the International Conference on Software Engineering (ICSE) 2013. Pages 742-751. ACM/IEEE.
    - This paper challenges a common assumption by programmers that efficiency trumps design. In particular, we challenged an extremely common "Iterator" design pattern to avoid breaking the encapsulation of the collection that iterators traverse. We justify our claim by an extensive and thorough study of the variations of encapsulation-friendly approaches applied to real world. Adopting more encapsulation-friendly approach to programming is known to result in both correctness benefits and advantages for parallelising code to run faster on modern multi-core machines. While such advantages were known for the last 10-15 years, we were the first to provide an extensive study that actually demonstrated it.

Employment Record

2022 – NOW	Associate Professor, Australian National University
2023 – NOW	Adjunct Professor, S3D, Carnegie Mellon University, USA
2025	Research Consultant, Usability Dynamics Inc, NC, USA.
2025	Research Consultant, Skykraft, Canberra, Australia.
2023 – 2025	Associate Director, HDR, School of Computing, ANU
2022 – 2025	Adjunct Professor, ECS, VUW, NZ
2022 – 2022	Deputy Head of School, Victoria University of Wellington
2021 – 2022	Associate Dean (Students), Victoria University of Wellington
2018 – 2022	Associate Professor, Victoria University of Wellington
2019/20 (sabbatical)	Associate Professor, Kyoto University, Japan
2010 – 2017	Senior Lecturer, Victoria University of Wellington
2013 (sabbatical)	Research Scholar, Carnegie Mellon University
2006 – 2009	Lecturer, Victoria University of Wellington
2005 – 2006	Software Engineer, Innaworks Limited

2004	Visiting Researcher, Purdue University
2001 – 2003	Software Engineer, Catalyst Systems Limited (part-time)

Professional Memberships

2024 – 2027, Member at Large (and Treasurer), *ACM SIGPLAN Executive Committee*
2023 – NOW, Member of Engineers Australia
2002 – NOW, Member of the ACM (currently: Senior Member of the ACM)
2018 – 2022, Member of Engineering New Zealand
2010 – 2014, Member of CORE (CS Departments of Australia/NZ) Executive

Grants and Awards

2026 – ESOP/ETAPS 2026 Distinguished Paper Award
2022 – FORTE 2022 Best Paper Award
2020 – 2021 – Robonomics.Network Research Grant ($72,000 NZD)
2021 – 2023 – SHEADI Faculty Strategic Initiative PhD Scholarship ($100,000 NZD)
2017 – 2018 – Oracle Corporation Research Grant ($70,000 NZD)
2017 – ECOOP 2017 Distinguished Artefact Award.
2014 – ECOOP 2014 Distinguished Paper Award.
2009 – 2011 – RSNZ Fast Start Marsden Grant ($300,000 NZD)
2012 – Mozilla Foundation Research Grant ($15,000 NZD)
2016, 2008, 2007 – VUW URF Grant ($18,000 NZD ’16, $12,000 NZD ’08, $17,000 NZD ’07)
2009 – RSNZ ISAT Grant ($5,420 NZD)
2007 – ESEC/FSE ACM SIGSOFT Distinguished Paper Award
2005 – 2nd Prize and the Judges Prize at the ICFP Programming Contest 2005 and 2003
2004 – Claude McCarthy Fellowship and 2003 – J. L. Stewart Scholarship
2003 – 2nd Place in the ACM International Research Competition (via OOPSLA2002 SRC)
2002 – Freemasons Scholarship and 2001 – Datacom Scholarship in Computer Science
1997 – Finalist in the George Soros International Mathematics Olympiad
1996 – Winner of the Russian City Tournament in Mathematics in 1996

Program or Review Committee Chair

2025: Asian Symposium on Programming Languages and Systems (APLAS) 2025.
2024: *Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)* – with Bur-Yuh Chang
2016: Asia-Pacific Software Engineering Conference (APSEC) – with Gail Murphy
2010, 2011: Computing: The Australasian Theory Symposium (CATS) – with Taso Viglas

General Chair

2022: SPLASH/OOPSLA 2022 (Object-Oriented Programming, Systems, Languages, and Applications) to be held in Auckland, New Zealand, combined with APLAS 2022, Dec’ 22.
2018: Asian Symposium on Programming Languages and Systems (APLAS) 2018, held in Wellington, NZ.
2015: New Object-Oriented Languages (NOOL) Workshop.
2009: Australasian Computer Science Week (ACSW) in Wellington, NZ, Jan ’09.
2004: 15th PhD Workshop and Doctoral Symposium at ECOOP 2005.
2004: 2nd International Dylan Conference. Wellington, NZ. 4-6 February 2004.

Program or Review Committee Member

OOPSLA 2021 (ERC), 2020, 2019, 2014, 2011 (ERC), APLAS 2021, 2016, ECOOP 2023, 2022, 2019, 2016, HOTSOS 2019, PLDI SRC 2019, GPCE 2019, NOOL 2016, FTfJP 2016, PLDI 2025, 2015 (ERC), FOOL 2014, ACSC 2010, 2013,

2015, 2017, IWACO 2024, 2022, 2011, 2007, FormaliSE 2022, 2023, Programming 2022, 2023, Onward! Papers 2023.

Organising Committee Member

Steering Committee Chair (from 2024) and Member (from 2019) of SPLASH/OOPSLA, Virtualisation Co-Chair for SPLASH/OOPSLA 2021, 2020, Publications Chair of SPLASH/OOPSLA 2015, 2016, 2017, Workshops Chair of SPLASH/OOPSLA 2017, 2018, Publications Chair of ICFP 2018.

Research

I am one of the leading experts internationally in the fields of object ownership (restricting access using graph theoretical properties) and immutability (precise definition of when objects are allowed to change). I have an international research track record in secure programming language design and type systems. My earlier ownership and immutability work benefitted established programming language designs such as Mozilla's Rust. One of my works on exploiting object graph structures uncovered security flaws in Java servers such as Jenkins/Tomcat and JBoss, as well as their equivalents for .NET.

In 2013 - 2023, my major research project was the Wyvern Secure Programming Language, which I co-led with Associate Professor Jonathan Aldrich (CMU). I bring expertise in formal verification and security. At the same time, Jonathan provided expertise in software architecture, which we applied to produce the first secure and verified software architecture support (we co-authored over a dozen papers as listed on the project web page: http://wyvernlang.github.io/).

In 2008 – 2012, I led the Unification of Immutability and Ownership project, which resulted in the Royal Society of NZ (equivalent of NSF in the US) Marsden Grant and 1 PhD and 3 Master's thesis students.

My research reputation is acknowledged by a past invitation to the Shonan Workshop and a full membership of the invitation-only IFIP Working Group 2.4, where a small number of leading international researchers present and discuss their latest work in an intensive week of interactions.

I have nearly 100 quality-assured publications as listed on my web page. I supervised over 80 students at different levels in the past, including 15 current ones (1 Postdoc, 10 (5 as primary) PhD, 3 Honours, 1 RA), as also listed on my web page.

Teaching

In my time at the ANU, I have created a new 2nd-year Software Engineering course that involves a group project consolidating the programming skills the students have and preparing them for the modern real-world environment. As the outcome of the course, each student contributes a Pull Request to an acceptable medium to large established open-source project.

When I was at the VUW, I got a consistent "overall effectiveness" score of around 1.6 (on a scale of 1 to 5, with one being "outstanding"). I taught classes in a variety of sizes (5-25 at the 4th year level, 60-120 at the 3rd year level, 150-300 at the 2nd year level, 300-500 at the 1st year level) and always received positive feedback with a "perfect" 1.0 score for "Attitude towards students". My overall teaching evaluation for the newly developed course on Software Development for Mobile Platforms in 2018 was the best possible 1.0. In several courses in 2020, I had close to 1.0 or equal to 1.0 (e.g., in SWEN 430 (Compiler Engineering) in 2020), teaching evaluations, despite it being the COVID year.

In 2019, I led the introduction of the postgraduate Artificial Intelligence and Machine Learning program at VUW as part of my role as Program Director (Science). This involved staff management and conflict resolution in a timely fashion to meet the required government regulation deadlines.

In 2017, I led a review of Software Engineering (the largest major in Bachelor of Engineering at the VUW) involving all the academic staff in Engineering and Computer Science. I led a discussion by over 20 software-related academic staff members over the first half of 2017, which led to a proposal I wrote that the VUW Academic Board approved in the middle of 2017. This is the first and most significant overhaul of the Software Engineering at the VUW since its creation 8 years ago. This will feed directly into our 5-yearly Engineering re-accreditation in 2017, which I also lead as the Program Director for Software Engineering.

Some specific examples of teaching innovation and course development include:

1. COMP 3500/4500/8715 TechLauncher re-design at the ANU. In 2025, I was tasked with a redesign of a capstone project course for Computing and Software Engineering graduates with 400 students. I changed the course to a scalable mode of operation able to run with 600+ student enrolments by focusing on the time-boxed prescribed model with teams formed around "the most available day" in their schedule and asking them to work in person in the shared "The Hive" space on campus with up to 15 "pods" available for the teams. At the same time, a tutor can oversee up to four teams simultaneously. We have reduced the tutor budget by half while increasing the student satisfaction and client satisfaction with deliverables significantly, and still fulfilling the relevant ACS and EA capstone requirements. *COMP3500 in 2025 S1 had a perfect 100% overall satisfaction score from the students.*
2. COMP 2120 re-design at the ANU. I took over the 2nd-year "Software Engineering" course. I revised both the assessment and content to create a modern, group project-oriented course with the ultimate goal of contributing a pull request to a large open-source project by every one of the 50+ teams.
3. SWEN 325 at the VUW – the creation of a brand-new course in 2018. To fill the gap in teaching our students high-level frameworks and software architecture as well as the in-demand skills in the development of software for mobile platforms and IoT, I developed from scratch a brand new final year undergraduate course that covers architecture for mobile systems, UX for mobile, being able to learn and work with multiple modern frameworks such as React Native and Ionic, and writing apps for and interacting with IoT devices. The course was very successful, with an overall teaching evaluation of me being 1.0 and a course overall evaluation of 1.8.
4. COMP 361 Full Course Redesign in 2014 at the VUW. I have addressed low enrolments in a 3rd-year "Algorithms" course (formerly known as COMP 303) by reinventing it as COMP361 with a readjusted aim and course objectives to make it more practical for modern software engineers. The redesign was a success: increasing the enrolments tenfold from 5-6 for COMP303 to around 60 from 2014 onwards. I am the sole person responsible for the course and its design and delivery. The final 2014 overall course feedback was 2.0, which is quite reasonable given that it originated from the smallest 3rd-year COMP course and has since become the second-largest after COMP 307 (Introduction to Artificial Intelligence) and the largest in the second Trimester for the 3rd-year COMP.
5. SWEN 302 Full Course Redesign in 2014 at the VUW. In 2013, SWEN302 ended up in need of a "rescue" when it received one of the worst course evaluations in ECS history: 4.0. I volunteered for the task as the course coordinator. I completely revamped the way this agile project course is run by bringing in mostly internal, very highly involved clients and introducing "time boxing": all the students were asked to pick one day every week that they had the least amount of lectures. The teams were then formed for those days of the week, and the requirement was that students should work only 8 hours on the designated day (in addition to 2 hours spent in a lecture). This worked wonders as it allowed students to control their time on one hand and to avoid any team members "slacking" as it was clear who had to be around when. More importantly, this made all the students learn the most essential aspect of agile management: being aware of the time spent on different parts of the project and being able to make accurate estimates at all times. The course received anecdotally positive feedback, and the overall score decreased significantly (to 2.0 in 2014).

6. COMP 261 at the VUW: Together with Dr Peter Andreae in 2010, we developed a brand-new course on Algorithms to fill the need for our up-and-coming Computer Graphics program, as well as providing an elective foundational course on Computer Algorithms for our Bachelor of Engineering students. The course was designed from the ground up to fit into the new 15-point system to keep the workload for students manageable. I put together a custom textbook with the help of the Pearson publisher that we use to teach COMP261. For the rest of my tenure at the VUW, COMP261 remained the most popular course in the undergraduate computer science degree.
7. SWEN 430 Full Course Redesign in 2010 at the VUW. Together with Dr David Pearce in 2010, we redesigned our graduate course to offer our up-and-coming Bachelor of Engineering students (especially Software Engineering majors) a strong course on compiler engineering and advanced Java programming. This course was a success: Innaworks and other Wellington start-ups sought after the students who took this course. In 2012, it was rated as the most useful course in course evaluations of the 4th-year Software Engineering courses.

Service

Since I started at the ANU in June 2022, I have been the program convenor for the Bachelor of Engineering in Software Engineering. From 2023, I also took on the role of the Diversity, Belonging, Inclusion, and Equity Working Group Lead for the School of Computing. Finally, from 2023 to 2025, for 2.5 years, I was the Associate Director, HDR for the School of Computing – looking after our cohort of just over 200 PhD students.

As part of my role as the Associate Director of HDR, I have created a monitoring app to track the progress and conditions of all of our HDR students in the school. The app is being adopted by the core administrative staff and key academics. I have revamped and optimised the processes, establishing a clear admissions and scholarship process that takes diversity into account and clarifies the roles of primary supervisors to ensure continuity for all our HDR students. I ensured that each of the four clusters has regular and consistent HDR monitoring in place every six months to improve the HDR cohort experience proactively.

At the very start of 2022, while still at the VUW, I was employed as Deputy Head of School as an additional 0.4 FTE on top of my other admin role described next to help with the transitional period as we are split into four sub-schools. I have 15 academic staff as direct reports, ranging from Assistant Lecturer to Full Professor, in the areas of Software Engineering and Cyber-Security. I have managed to organise for all of them to conduct annual performance development plans for the year within a week of starting my role. One of the main reasons I have been asked to fill this role was because I managed to both accredit and revise our Bachelor of Engineering in Software Engineering during my tenure as Program Director (described below) while keeping all the academics involved both engaged and happy with the proposed changes and getting them through all the necessary academic proposal stages, including VUW Academic Board, and eventual approval by the NZ national CUAP committee, as well as the scheduled accreditation by the Engineering NZ.

In 2021 and 2022 at the VUW, I was employed as 0.4 FTE Associate Dean (Students) at the Faculty of Engineering, looking after approximately 1600 EFTS (or 2000 actual students, as some are part-time) in the School of Engineering and Computer Science and the School of Mathematics and Statistics. One of my first achievements was the creation of the Student Engagement System that allows me to proactively monitor the progress of ~2000 students in our faculty: picking up those who are asking for extensions, missing deadlines, or not coping with the courses that they are currently enrolled in. With the help of the Faculty Office staff, we can reach out early to them before they become non-engaged and place appropriate support measures in place, ranging from connecting them with counselling services to offering withdrawals and re-weighting of the assessments. Alongside the Student Engagement System (SES), I have also centralised our assessment scheduling, which is now done proactively before each trimester begins because the academic staff saw the clear benefits of the SES in picking up struggling students early and were willing to provide the relevant information early to help track the progress and help the students succeed. Finally, our extension requests are now integrated into our Assessment System to ensure consistency (especially in times of COVID) and the ability

to easily monitor the student's progress. Below is the screenshot with no names showing one of the tabs in the system tracking the students and sorting by the "concern score". Finally, one of the examples of numerous examples of external engagement was my negotiation in January 2022 of a special pathway from a regional polytechnic (WITT) that allows the students doing their first year locally as BEngTech in the regional NZ (e.g. still staying with their parents in the time of Covid) to then do our summer papers in COMP remotely in the summer to join alongside our 2nd year students to be able to complete BE at VUW – easing the transition from the regions to the far away city University study.

In 2017 – 2018, while at the VUW, I served as Program Director and led the Software Group, delivering the SWEN, NWEN, and CYBR majors in BE and the HSWD major in Bachelor of Health, which involves working with over 20 academic and other teaching staff and assisting the Head of School in running these programs. A major part of this role is to enable staff to work productively and constructively together to deliver the best learning experience to students. In 2019, I moved to the Program Director (Science) role to lead the introduction of the AI & ML postgraduate program before leaving on a partial sabbatical in 2020.

As Postgraduate Coordinator from 2012 to 2016 while at the VUW, I led an interdisciplinary postgraduate research program with nearly 100 PhD and Master's students, chaired every PhD proposal meeting, and attended to all matters related to thesis students. I significantly clarified and improved the procedures for induction, reporting, handling of problem cases, PhD progression to full registration, Master's Thesis progress reporting and examinations at the Engineering Faculty during my term, which was my key responsibility. My work earned praise from the FGR and received a commendation from the BE Review Panel in 2016. When I finished, my responsibilities were handed over to three Associate Professors and a professional staff member.

In 2018, I was part of the Engineering New Zealand Accreditation Panel to re-accredit the University of Canterbury's Bachelor of Engineering with nine majors, and I represented the NZ Qualifications Authority to accredit a new Bachelor of Information Technology degree proposed by Whitireia and WelTec Polytechnics. I remained an annual monitor for the "W and W" IT programs at postgraduate and undergraduate levels for 2020, 2021, and 2022.

Finally, I was a member of the [senior] research track program committees for many international conferences including top ranked: OOPSLA (2021, 2020, 2019, 2014, 2011) and ECOOP (2019, 2016) and many others as listed on my web page as well as the General Chair and Review Committee Chair for the main conference in my area (OOPSLA) in 2022 and 2024.

Courses Taught

NB! 1st-year courses are usually in the 400 or so size range. The 2nd-year level is in the 300 or so range. The 3rd-year level is in the 100-200 or so range. The 4th-year level is around 20-30 people. Tech Launcher is 400 students (or 60 teams with external clients).

At the ANU:

1. COMP3500/4500/8715 Tech Launcher Capstone Project in 2025
2. COMP 2120 Semester 2 of 2024 (Software Engineering)
3. COMP 2100 Semester 2 of 2024 (Software Construction) – convening only
4. COMP 4130 Semester 1 of 2024 (Managing Software Quality and Process) – convening only
5. COMP 2120 Semester 2 of 2023 (Software Engineering)
6. COMP 2120 Semester 2 of 2022 (Software Engineering)

At the VUW:

7. COMP 103 T3 (Jan/Feb) 2022 (Introduction to Data Structures and Algorithms)
8. COMP 102 T3 (Summer, Half) 2021 Nov/Dec (Introduction to Computer Program Design)
9. COMP 261 T1 (Second Half) 2021 (Algorithms and Data Structures)
10. COMP 361 T1 (First Half) 2021 (Design and Analysis of Algorithms)

11. COMP 103 T3 (Second Half) 2020 (Introduction to Data Structures and Algorithms)
12. SWEN 430 T2 (Second Half) 2020 (Compiler Engineering)
13. SWEN 325 T2 (Weeks 2, 3, 4) 2020 (Software Development for Mobile Platforms)
14. Kyoto University Graduate Seminars in Winter 2019/2020
15. SWEN 325 T2 2019 (Software Development for Mobile Platforms)
16. SWEN 325 T2 2018 (Software Development for Mobile Platforms)
17. COMP 361 T2 2017 (Design and Analysis of Algorithms)
18. COMP 261 T1 (Course Coordinator Only) 2017 (Algorithms and Data Structures)
19. SWEN 221 T1 (Course Coordinator Only) 2017 (Software Development)
20. COMP 103 T2 (Coordinator Only) 2016 (Intro to Data Structures and Algorithms)
21. COMP 361 T2 2016 (Design and Analysis of Algorithms)
22. COMP 261 T1 (9 of 12 weeks) 2016 (Algorithms and Data Structures)
23. COMP 361 T2 2015 (Design and Analysis of Algorithms)
24. SWEN 302 T2 (Essays and Course Coordinator Only) 2015 (Agile Methods)
25. ENGR 123 T2 (Labs Only) 2015 (Engineering Mathematics with Logic and Statistics)
26. COMP 261 T1 (First Eight Weeks) 2015 (Algorithms and Data Structures)
27. COMP 361 T2 2014 (Design and Analysis of Algorithms)
28. SWEN 302 T2 (Coordinator Only) 2014 (Agile Methods)
29. COMP 261 T1 (Second Half) 2014 (Algorithms and Data Structures)
30. SWEN 223 T1 (First Half) 2014 (Software Engineering Analysis)
31. SWEN 430 T1 (First Half) 2014 (Compiler Engineering)
32. COMP303 T2 2012 (Design and Analysis of Algorithms)
33. SWEN430 T2 2012 (Compiler Engineering)
34. COMP103 T1 2012 (Introduction to Data Structures and Algorithms)
35. COMP303 T2 2011 (Design and Analysis of Algorithms)
36. COMP261 T2 (Last 8 weeks) 2011 (Algorithms and Data Structures)
37. SWEN430 T2 (First 4 weeks) 2011 (Compiler Engineering)
38. SWEN423 T1 (First 2 weeks and last 4 weeks) 2011 (OO Paradigms)
39. COMP303 T2 2010 (Design and Analysis of Algorithms)
40. COMP261 T2 (2nd half) 2010 (Algorithms and Data Structures)
41. SWEN430 T2 (1st half) 2010 (Compiler Engineering)
42. COMP303 T2 (2nd half) 2009 (Design and Analysis of Algorithms)
43. COMP431 T2 (2nd half) 2009 (Compilers)
44. COMP304 T1 (1st half) 2009 (Programming Languages)
45. COMP462 T1 (1st half) 2009 (Object-Oriented Paradigms)
46. COMP303 T2 2008 (Design and Analysis of Algorithms)
47. COMP431 T1 2008 (Compilers)
48. COMP103 T2 (2nd half) 2007 (Introduction to Data Structures and Algorithms)
49. COMP471 T2 (1st half) 2007 (Special Topic: Compiler Technologies)
50. COMP462 T1 (1st half) 2007 (Object-Oriented Paradigms)
51. COMP103 T3 (2nd half) 2006 (Introduction to Data Structures and Algorithms)
52. COMP103 T2 (2nd half) 2006 (Introduction to Data Structures and Algorithms)
53. COMP101 T3 2005 (Introduction to Dynamic Web Development)
54. INET101 T3 2004 (Introduction to Internet Technology)

Full Publications List

Book Chapters

1. James Noble, Alex Potanin, Toby Murray, Mark S. Miller. Abstract and Concrete Data Types vs Object Capabilities. In P. Müller and Ina Schaefer (Eds.): Principled Software Development - Essays Dedicated to Arnd Poetzsch-Heffter on the Occasion of his 60th Birthday. Springer, Heidelberg. 2018.

2. Alex Potanin, Johan Ostlund, Yoav Zibin, Michael D. Ernst. Immutability. In D. Clarke et al. (Eds.): Aliasing in Object-Oriented Programming, LNCS 7850, pp. 233-269. Springer, Heidelberg. 2013.

Journal Papers

3. Zara Hassan, Christoph Treude, Michael Norrish, Graham Williams, and Alex Potanin. Characterising Reproducibility Debt in Scientific Software: A Systematic Literature Review. The Journal of Systems & Software. Volume 222, April 2025, 112327.
4. Tobias Runge, Tabea Bordis, Alex Potanin, Thomas Thum, Ina Schaefer. Flexible Correct-by-Construction Programming. Logical Methods in Computer Science (Accepted in April 2023).
5. Tobias Runge, Marco Servetto, Alex Potanin, Ina Schaefer. Immutability and Encapsulation for Sound OO Information Flow Control. ACM Transactions on Programming Languages and Systems. Volume 45, Issue 1, Article No.: 3 pp 1–35.
6. Isaac Oscar Gariano, Marco Servetto, Alex Potanin. Using Capabilities for Strict Runtime Invariant Checking. Science of Computer Programming. (Accepted in September 2022).
7. Darya Melicher, Anlun Xu, Valerie Zhao, Alex Potanin, Jonathan Aldrich. Bounded Abstract Effects. ACM Transactions on Porgramming Languages and Systems, Volume 44, Issue 1. March 2022. Article No 5, pp 1 – 48.
8. Isaac Oscar Gariano, Marco Servetto, Alex Potanin, Hrshikesh Arora. Iteratively Composing Statically Verified Traits. VPT-2019 Post-Proceedings as a volume of Electronic Proceedings in Theoretical Computer Science (EPTCS). Issue 299, Paper 7.
9. Chris Male, David Pearce, Alex Potanin, and Constantine Dymnikov. Formalisation and Implementation of an Algorithm for Bytecode Verification of @NonNull Types. Science of Computer Programming. Volume 76, Issue 7, Pages 587 - 608, July 2011.
10. Alex Potanin, James Noble, Dave Clarke, and Robert Biddle. Featherweight Generic Confinement. Journal of Functional Programming. Volume 16, Number 6, Pages 793 - 811, September 2006.
11. Alex Potanin, James Noble, Marcus Frean, and Robert Biddle. Scale-free Geometry in Object-Oriented Programs. Communications of the ACM. Pages 99 - 103. May 2005. (ACM Link)
12. Alex Potanin, James Noble, and Robert Biddle. Checking Ownership and Confinement. Concurrency and Computation: Practice and Experience. Volume 16, Issue 7, Pages 671 - 687, 2004.

Refereed Conference Papers

13. Liyi Li, Anshu Sharma, Zoukarneini Difaizi Tagba, Sean Frett, and Alex Potanin. Validating Quantum State Preparation Programs. In ESOP 2026 (Accepted in Round 1).
14. Feifei Cheng, Sushen Vangeepuram, Henry Allard, Seyed mohammad reza Jafari, Alex Potanin, and Liyi Li. Embedding Quantum Program Verification into Dafny. In OOPSLA 2025.
15. Maximilian Kodetzki, Tabea Bordis, Alex Potanin, Ina Schaefer. X-by-Construction: Towards Ensuring Non-Functional Properties in by-Construction Engineering. In Onward! 2025.
16. Emmanuel Iko-Ojo Simon, Chirath Hettiarachchi, Alex Potanin, Hanna Suominen, and Fatemeh Fard. Automated Detection of Algorithm Debt in Deep Learning Frameworks: An Empirical Study. In ICSME 2024 Registered Reports Track.
17. David Young, Ziyi Yang, Ilya Sergey, Alex Potanin. Higher-Order Specifications for Deductive Synthesis of Programs with Pointers. In ECOOP 2024.
18. Amos Robinson, Alex Potanin. Pipit on the Post: proving pre- and post-conditions of reactive systems. In ECOOP 2024.
19. Zara Hassan, Christoph Treude, Michael Norrish, Graham Williams, Alex Potanin. Reproducibility Debt: Challenges and Future Pathways. In FSE-IVR 2024 (FSE 2024 Ideas, Visions and Reflections Track).
20. Tobias Runge, Alexander Kittelmann, Marco Servetto, Alex Potanin, and Ina Schaefer. Information Flow Control-by-Construction for an Object-Oriented Language. In proceedings of SEFM 2022.
21. Tobias Runge, Alex Potanin, Thomas Thum, and Ina Schaefer. Traits: Correctness-by-Construction for Free. In proceedings of FORTE 2022. Won FORTE 2022 Best Paper award.

22. Manish Singh, Lindsay Groves, and Alex Potanin. A Relaxed Balanced Lock-free Binary Tree. In proceedings of PDCAT 2020.
23. Julian Mackay, Alex Potanin, Jonathan Aldrich, and Lindsay Groves. Syntactically Restricting Bounded Polymorphism for Decidable Subtyping. In proceedings of APLAS2020.
24. Julian Mackay, Alex Potanin, Jonathan Aldrich, and Lindsay Groves. Decidable Subtyping for Path Dependent Types. In proceedings of POPL2020. Additionally, the artifact successfully passed the Artifact Evaluation and was given Reusable Artifact badge.
25. Aaron Craig, Alex Potanin, Lindsay Groves and Jonathan Aldrich. Capabilities: Effects for Free. In proceedings of ICFEM2018. Pp 231-247. Springer.
26. Jens Dietrich, Kamil Jezek, Shawn Rasheed, Amjed Tahir, Alex Potanin. EvilPickles: DoS attacks based on Object-Graph Engineering. In proceedings of ECOOP2017. Additionally, the artifact successfully passed the Artifact Evaluation. This paper won ECOOP 2017 Distinguished Artifact Award.
27. Darya Melicher, Yangqingwei Shi, Alex Potanin, Jonathan Aldrich. A Capability-Based Module System for Authority Control. In proceedings of ECOOP2017. Additionally, the artifact successfully passed the Artifact Evaluation.
28. Garming Sam, Nicholas Cameron and Alex Potanin. Automated Refactoring of Rust Programs. Article No.: 14. In proceedings of ACSC2017.
29. Joseph Lee, Jonathan Aldrich, Troy Shaw, Alex Potanin. A Theory of Tagged Objects. In proceedings of ECOOP2015. Pp 999-1026. Additionally, the artifact on the software section of my home page successfully passed the Artifact Evaluation.
30. Cyrus Omar, Darya Melicher, Ligia Nistor, Benjamin Chung, Alex Potanin, and Jonathan Aldrich. Safely Composable Type-Specific Languages. In proceedings of ECOOP2014. Pp 105-130. Springer-Verlag. This paper won ECOOP 2014 Distinguished Paper Award.
31. Marco Servetto, Julian Mackay, Alex Potanin, and James Noble. The Billion-Dollar Fix: Safe Modular Circular Initialisation with Placeholders and Placeholder Types. In proceedings of ECOOP2013. Pp 205-229. Springer-Verlag.
32. Constantine Dymnikov, David Pearce and Alex Potanin. OwnKit: Inferring Modularly Checkable Ownership Annotations for Java. In proceedings of the Australasian Software Engineering Conference (ASWEC) 2013. Pp 181-190. IEEE.
33. Alex Potanin, Monique Damitio and James Noble. Are Your Incoming Aliases Really Necessary? Counting the Cost of Object Ownership. In proceedings of the International Conference on Software Engineering (ICSE) 2013. Pp 742-751. ACM/IEEE.
34. Daniel Atkins, Alex Potanin and Lindsay Groves. The Design and Implementation of Clocked Variables in X10. In proceedings of Australasian Computer Science Conference (ACSC) 2013. Pp 87-96. CRPIT.
35. Jan Larres, Alex Potanin and Yuichi Hirose. A Study of Performance Variations in the Mozilla Firefox Web Browser. In proceedings of Australasian Computer Science Conference (ACSC) 2013. Pp 3-12. CRPIT.
36. Hien Tran, Craig Anslow, Stuart Marshall, Alex Potanin, Mairead de Roiste. Lessons Learnt from Collaboratively Creating Maps on a Touch Table. In proceedings of the 12th Annual Conference of the New Zealand Chapter of the ACM Special Interest Group on Computer-Human Interaction (CHINZ) 2011. Pp. 105-108. ACM SIGCHI.
37. Yoav Zibin, Alex Potanin, Paley Li, Mahmood Ali, Michael D. Ernst. Ownership and Immutability in Generic Java. In proceedings of Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA) 2010. Pp. 598-617. ACM.
38. Radu Muschevici, Alex Potanin, Ewan Tempero, and James Noble. Multiple Dispatch in Practice. In proceedings of Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA) 2008. Pp. 563-582. ACM. (ACM Link)
39. Chris Male, David Pearce, Alex Potanin, and Constantine Dymnikov. Java Bytecode Verification for @NonNull Types. In proceedings of Compiler Construction (CC) 2008. Pp 229-244.
40. Neil Ramsay, Stuart Marshall, and Alex Potanin. Annotating UI Architecture with Actual Use. In proceedings of Australasian User Interface Conference (AUIC) 2008. Pp. 75-78. CRPIT.

41. Yoav Zibin, Alex Potanin, Mahmood Ali, Shay Artzi, Adam Kiezun, and Michael D. Ernst. Object and Reference Immutability using Java Generics. In proceedings of the Foundations of Software Engineering (FSE) Conference, Dubrovnik, Croatia, September 2007. Pp. 75-84. ACM. (Won ESEC/FSE 2007 ACM SIGSOFT Distinguished Paper Award.) (ACM Link)
42. Alex Potanin, James Noble, Dave Clarke, and Robert Biddle. Generic Ownership for Generic Java. In proceedings of the Object-Oriented Programming, Systems, Languages and Applications (OOPSLA) Conference, Portland, Oregon, October 2006. Pp. 311-324. ACM. (ACM Link)
43. Alex Potanin, James Noble, and Robert Biddle. Snapshot Query-Based Debugging. In proceedings of the Australian Software Engineering Conference, Melbourne, Australia, April 2004. Pp 251-261.

Refereed Workshop Papers

44. Sasha Pak, Fabian Muehlboeck, Alex Potanin. A Verified Thread-Safe Array in Rust. In IWACO 2025.
45. Abhaas Goyal, Alex Potanin and Jonathan Aldrich. A Comparative Study of Traditional versus Capability-Based Module Systems for Modern Programming Languages. In PLATEAU 2024.
46. Amos Robinson and Alex Potanin. Pipit: Reactive Systems in F Star (Extended Abstract). In TyDe 2023.
47. Baptiste Pauget, David Pearce, and Alex Potanin. Towards Compilation of an Imperative Language for FPGAs. In VMIL 2018.
48. James Noble, Sophia Drossopoulou, Mark S Miller, Toby Murray and Alex Potanin. Abstract Data Types in Object-Capability Systems. In IWACO 2016.
49. Du Li, Alex Potanin, and Jonathan Aldrich. Delegation vs Inheritance for Typestate Analysis. In FTfJP 2015.
50. Darya Melicher, Alex Potanin, and Jonathan Aldrich. Wyvern: Impacting Software Security via Programming Language Design. In PLATEAU 2014. Pages 57-58.
51. James Noble and Alex Potanin. On Owners-as-Accessors. In IWACO 2014.
52. Jonathan Aldrich, Cyrus Omar, Alex Potanin and Du Li. Language-Based Architectural Control. In IWACO 2014.
53. Cyrus Omar, Benjamin Chung, Darya Kurilova, Alex Potanin and Jonathan Aldrich. Type-Directed, Whitespace-Delimited Parsing for Embedded DSLs. In proceedings of the First Workshop on the Globalization of Domain Specific Languages (GlobalDSL) 2013. Pp 8-11.
54. Ligia Nistor, Darya Melicher, Stephanie Balzer, Benjamin Chung, Alex Potanin and Jonathan Aldrich. Wyvern: A Simple, Typed, and Pure Object-Oriented Language. In Proceedings of the 5th Workshop on MechAnisms for SPEcialization, Generalization and inHeritance (MASPEGHI) 2013. Pp 9-16.
55. Marco Servetto, David Pearce, Lindsay Groves, and Alex Potanin. Balloon Types for Safe Parallelisation over Arbitrary Object Graphs. Workshop on Determinism and Correctness in Parallel Programming (WoDeT) 2013.
56. Daniel Atkins, Alex Potanin, Lindsay Groves. Clocked References in X10. Languages for the Multicore Era (LaME) 2012.
57. Julian Mackay, Hannes Mehnert, Alex Potanin, Lindsay Groves, Nicholas Cameron. Encoding Featherweight Java with Assignment and Immutability using The Coq Proof Assistant. In the proceedings of Formal Techniques for Java-like Programs (FTfJP) 2012. Pp 11-19.
58. Yoav Zibin, Alex Potanin, Paley Li, Mahmood Ali, Michael D. Ernst. Ownership and Immutability in Generic Java (OIGJ). IBM Programming Languages and Development Environments Seminar 2010 (PLDE2010).
59. Paley Li, Stephen Nelson, and Alex Potanin. Ownership for Relationships. International Workshop on Aliasing, Confinement, and Ownership (IWACO) 2009. Article No. 8.
60. Paley Li, Alex Potanin, James Noble, and Lindsay Groves. Towards Unifying Immutability and Ownership. International Workshop on Aliasing, Confinement, and Ownership (IWACO) at ECOOP2008.
61. Christo Fogelberg, Alex Potanin, and James Noble. Ownership Meets Java. International Workshop on Aliasing, Confinement, and Ownership (IWACO) at ECOOP2007.

62. Alex Potanin, James Noble, Tian Zhao, Jan Vitek. A High Integrity Profile for Memory Safe Programming in Real-time Java. In proceedings of the 3rd workshop on Java Technologies for Real-time and Embedded Systems, San Diego, CA, USA, October 2005.
63. Alex Potanin, James Noble, Dave Clarke, Robert Biddle. Featherweight Generic Ownership. In proceedings of the workshop on Formal Techniques for Java-like Programs (FTfJP) in European Conference for Object-Oriented Programming, Glasgow, Scotland, July 2005.
64. Alex Potanin, James Noble, Dave Clarke, Robert Biddle. Defaulting Generic Java to Ownership. In proceedings of the workshop on Formal Techniques for Java-like Programs (FTfJP) in European Conference for Object-Oriented Programming, Oslo, Norway, June 2004.
65. Alex Potanin, James Noble, Dave Clarke, and Robert Biddle. Featherweight Generic Confinement. In proceedings of the International Workshop on Foundations of Object-Oriented Languages (FOOL), affiliated with Symposium on Principles of Programming Languages, Venice, Italy, January 2004.
66. James Noble, Robert Biddle, Ewan Tempero, Alex Potanin, and Dave Clarke. Towards a Model of Encapsulation. In proceedings of the International Workshop on Aliasing, Confinement and Ownership (IWACO) in European Conference for Object-Oriented Programming, Darmstadt, Germany, July 2003.
67. Alex Potanin and James Noble. Checking Ownership and Confinement Properties. In proceedings of the workshop on Formal Techniques for Java-like Programs (FTfJP) in European Conference for Object-Oriented Programming, Malaga, Spain, June 2002.

Edited Journals

68. Alex Potanin and Bor-Yuh Evan Chang (Associate Editors). OOPSLA 2024 PACMPL Volume 8 Issue OOPSLA1 and PACMPL Volume 8 Issue OOPSLA2.
69. Alex Potanin and Gail Murphy (Editors). Special Issue on the 23rd Asia-Pacific Software Engineering Conference (APSEC) 2016. Science of Computer Programming. Volume 163, 1 October 2018.
70. Alex Potanin (Editor). Special Issue on New Object-Oriented Languages (NOOL) 2015. Journal of Object Technology. Volume 16, no. 2 (April 2017).
71. Taso Viglas and Alex Potanin (Editors). Special Issue on Computing: The Australasian Theory Symposium 2011. International Journal on Foundations of Computer Science (IJFCS), 2013. Vol. No. 24, Issue No. 1.
72. Taso Viglas and Alex Potanin (Editors). Special Issue on Computing: The Australasian Theory Symposium 2010. Chicago Journal of Theoretical Computer Science (CJTCS), May 2011. Volume 2010.

Other Refereed Publications

73. Sasha Pak, Richard Willie, Umang Mathur, Fabian Muehlboeck, Alex Potanin. View Types in Rust. Poster at SPLASH 2025.
74. Matthew Britton, Alex Potanin, Sasha Pak. Verifying Extract Method Refactoring in Rust. Poster at SPLASH 2025.
75. Zara Hassan, Christoph Treude, Graham Williams, Michael Norrish, Alex Potanin. Reproducibility Debt in Scientific Software. Poster at SPLASH 2025.
76. Manish Singh, Lindsay Groves, Alex Potanin. A Relaxed Balanced Non-Blocking Binary Search Tree. Poster in International Conference on Parallel Processing (ICPP) 2019.
77. Isaac Oscar Gariano, Marco Servetto, Alex Potanin, Hrshikesh Arora. Iteratively Composing Statically Verified Traits. Extended Abstract at Seventh International Workshop on Verification and Program Transformation (VPT 2019).
78. Darya Melicher, Yangqingwei Shi, Valerie Zhao, Alex Potanin, and Jonathan Aldrich. Using Object Capabilities and Effects to Build an Authority-Safe Module System. Poster in HotSoS 2018.
79. Aaron Craig, Alex Potanin, Lindsay Groves, Jonathan Aldrich. Capabilities and Effects. In OCAP 2017.

80. Darya Melicher, Yangqingwei Shi, Valerie Zhao, Alex Potanin, and Jonathan Aldrich. Using Object Capabilities and Effects to Build an Authority-Safe Module System. In OCAP 2017.
81. Jonathan Aldrich and Alex Potanin. Usably Expressing and Enforcing Design in Wyvern. In NOOL 2017.
82. Jens Dietrich, Kamil Jezek, Shawn Rasheed, Amjed Tahir, Alex Potanin. EvilPickles: DoS attacks based on Object-Graph Engineering (Artifact). In DARTS.
83. Darya Melicher, Yangqingwei Shi, Alex Potanin, Jonathan Aldrich. A Capability-Based Module System for Authority Control (Artifact). In DARTS.
84. Jonathan Aldrich and Alex Potanin. Delegation Revisited. NOOL 2016.
85. Jonathan Aldrich and Alex Potanin. Naturally Embedded DSLs. DSLDI 2016.
86. Darya Melicher, Alex Potanin, and Jonathan Aldrich. Modules in Wyvern: Advanced Control over Security and Privacy. Poster in HotSOS 2016. See last page here.
87. Joseph Lee, Jonathan Aldrich, Troy Shaw, and Alex Potanin. A Theory of Tagged Objects (Artifact). DARTS, Volume 1, Issue 1, 2015.
88. Cyrus Omar, Darya Melicher, Ligia Nistor, Benjamin Chung, Alex Potanin, and Jonathan Aldrich. Safely Composable Type-Specific Languages. Poster in ECOOP2014.
89. Darya Melicher, Cyrus Omar, Ligia Nistor, Benjamin Chung, Alex Potanin, Jonathan Aldrich. Type-Specific Languages to Fight Injection Attacks. Poster in HotSOS 2014.
90. Cyrus Omar, Benjamin Chung, Darya Melicher, Ligia Nistor, Alex Potanin, Jonathan Aldrich. Extensible Type-Driven Parsing for Embedded DSLs in Wyvern. Parsing@SLE 2013.
91. Cyrus Omar, Benjamin Chung, Darya Melicher, Ligia Nistor, Alex Potanin, Jonathan Aldrich. Extensible Type-Driven Parsing for Embedded DSLs in Wyvern. Poster in SPLASH2013.
92. Jonathan Aldirch, Benjamin Chung, Darya Melicher, Cyrus Omar, Alex Potanin. DSL support in Wyvern Language. Domain Specific Language Design and Implementation (DSLDI) 2013.
93. Jan Larres, Alex Potanin, and Yuichi Hirose. Performance Variance Evaluation on Mozilla Firefox. In the proceedings of the NZCSRSC2011.
94. Mairead de Roiste, Hien Tran, and Alex Potanin. What makes a map? At Ireland Organisation for Geographic Information Conference (IRLOGI) 2010.
95. Chris Andreae, Donald Gordon, Alex Potanin, James Noble, Robert Biddle. Terrier: Static Query-Based Debugging in Eclipse. Poster in Conference on Object-Oriented Programming, Systems, Languages, and Applications, Vancouver, Canada, October 2004.
96. Alex Potanin. Generic Ownership: Practical Ownership Control in Programming Languages. In Doctoral Symposium Proceedings of Conference on Object-Oriented Programming, Systems, Languages, and Applications, Vancouver, Canada, October 2004.
97. Alex Potanin. Practical Ownership Control in Programming Languages. In Doctoral Symposium Proceedings of European Conference for Object-Oriented Programming, Oslo, Norway, June 2004.
98. Alex Potanin. A Tool for Ownership and Confinement Analysis of the Java Object Graph. Conference poster and student research competition entry in Object-Oriented Programming Systems, Languages, and Applications Conference, Seattle, USA, November 2002. The poster itself is available in Microsoft PowerPoint format here. This entry was awarded a second place in the competition in the graduate division, even though I was technically an undergraduate. This entry was accepted into the ACM Student Research Competition Grand Finals and was awarded a second place overall in the undergraduate category as can be found here.

Theses

- Alex Potanin. Generic Ownership - A Practical Approach to Ownership and Confinement in OO Programming Languages. My PhD thesis, awarded in 2007.
- Alex Potanin. The Fox - A Tool for Object Graph Analysis. Honours report which resulted in First Class Honours at Victoria University of Wellington in 2002. My supervisors were James Noble and Robert Biddle.